Model card: credit scorecard acb-sc-1.0.0

Model inventory entry for the Agent Credit Bureau (spec 4.7). One entry per model id; a change to a formula is a new model id and a new module, a change to the configuration is a new `configHash`.

Field	Value
Model id	`acb-sc-1.0.0` (bytes32 short string on-chain)
Implementation	`src/acb/bureau/models/sc_1_0_0.py`, function `score(inputs, cfg, facts) -> Bundle` (pure, integer arithmetic)
Configuration	`config/scorecard.yaml`; `configHash = chash({"scorecard": <file>, "anchors": <deployment anchors>})`; archived as `config/scorecard/archive/<configHash>.json`
Detector used for verified reports	`acb-inj-1.0.0` (`src/acb/injection.py`), named in the configuration as `detector_id`
Owner	Team Mongolia (bureau package WP4); independent validation by the engineer who did not write the scorecard
Type	Expert-judgement (judgemental) additive scorecard; points are not log-odds calibrated
Status	Demo profile active for the FinTechathon 2026 run (Base Sepolia testnet); production profile defined, not used

Purpose

Grade software agents (ERC-8004 identities) as counterparties for payments by AI wallets. The grade (`A`, `B`, `C`, `D`, or `NEW` for insufficient history) gates what a wallet vault may pay without escrow or owner approval. The bureau scores agents only; it processes no data about natural persons.

Inputs (all public, read at or before `asOfBlock`)

- Orders in registered, config-bound `TierVault`s (`Paid`, `EscrowOpened` and their closing events), counted only if the amount is at least 0.10 USDC, a payee-signed invoice whose hash equals the on-chain `invoiceHash` is published (signer = the payee's `agentWallet` at the order's block, matching buyer vault, issuer and amount), and payer and payee are not in the same cluster.
- Clusters: agentIds that share an identity owner or a vault owner (union-find).
- Latest posted grades of counterparties and reporters (no recursion), and the configured anchors.
- ERC-8004 reports (`acb-report` feedback). Only verified accusations carry weight: INJECTION_ATTEMPT with an invoice the subject signed, a matching on-chain refusal and an adverse detector result; SPAM with three unsolicited invoices the subject signed; BAD_DATA backed by a settled, counted order of the reporter's vault. Praise and unbacked accusations weigh 0.
- Account age from the identity registry's `Registered` event and block timestamps.

Owners and wallets over time are derived from `Transfer`, `MetadataSet("agentWallet")`, `VaultInitialized`, `OwnershipTransferred` and `AgentChanged` events, so anyone can recompute a grade from logs and block headers (`acb bureau recompute`), without an archive node.

Output

An evidence bundle `acb-evidence/1` (canonical JSON) with every order, report, feature, point, rule and reason behind the grade. Its keccak256 (`evidenceHash`), the model id and the `configHash` are posted on-chain with the grade and mirrored to ERC-8004 Reputation as `acb-grade` feedback (value = score).

Method (summary; the normative text is spec section 4)

Features F1 fulfilment rate with a one-order prior at 80%, F2 failures, clawbacks and bad data, F3 flagged behaviour, F4 experience and account age, F5 counterparty diversity, quality by volume and circular volume (payment graph over clusters, three hops), F6 volume spike. Each feature maps to points by a monotone bin table; `score = clamp(600 + sum, 300, 850)`. Grade rules in order: hard flag on a verified injection, cold start below three settled orders, score bands (A 700, B 640, C 560), circular volume cap, standing gate (A or B needs a trade with a counterparty of standing 800 or more), age gate for A. Up to four adverse-action reason codes, rule codes first.

Positive evidence is weighted by the counterparty's standing (anchor and A 1000, B 800, NEW 250, C 200, NONE and D 0); adverse evidence only by counterparties in good standing (anchor and A 1000, B 800, all others 0). Failures are weighted by amount up to 0.20 USDC and capped at 3000 per payer cluster; reports are capped at 1000 per reporter cluster, subject and kind, deduplicated per reporter, order and kind, and decay after 30 days.

Validation

- Golden vectors G1-G14 of spec 4.5 are tests (`tests/bureau/test_bureau_golden.py`); scores, grades and reason codes match the spec table exactly.
- Monotonicity: 5,000 seeded random cases check that the score never falls when on-time weight, experience, age, diversity or counterparty quality rises, and never rises when late share, failures, disputes, flags, circular volume or spikes rise.
- Every bin boundary, reason ordering, the reporter rules, the counted-order filters and the fail cap have their own tests; on Anvil every posting recomputes to `MATCH` and the ERC-8004 summary equals the latest score.
- Backtest (`acb bureau backtest --agents 300 --rounds 6 --seed 2026`, spec 4.12): synthetic population of honest, slow, defaulting, bust-out, spamming, hijacked, dust-ring, wash-ring and smear-sybil agents. Result of the reference run (deterministic for the seed, about 11 s):

Metric	Value
AUC, good vs bad providers (final round)	0.9758
KS	0.8863
Median rounds from first default to grade D (defaulters)	1 (19 of 22 that defaulted reached D; 3 with few failures did not)
False adverse action rate (honest agents graded C or D, final round)	0.00%
Score change of honest providers from NEW-sybil failures and smears	0 in all 36 cases
Best grade ever reached by dust-ring and wash-ring members	C / NEW (never A or B)
PSI between consecutive rounds	0.86, 1.02 (bootstrap), then 0.08, 0.013, 0.012

The file `evidence/backtest.json` is produced by the scenario export (WP9) from the same command.

Limitations and known risks

- Expert-judgement points, not fitted to real default data; discrimination is measured on synthetic populations only.
- Anchors (verified data furnishers) are chosen by the bureau operator and bound by the config hash (KR-6). In the demo the only anchor is Beacon Research, a simulated buyer.
- Delivery outcomes of direct payments are reported by buyers (KR-3); only buyers in good standing have adverse weight, and the fail cap bounds a colluding cluster (KR-4: funding-source clustering is not implemented).
- Orders count only with a published invoice: lost evidence files remove orders from scores (KR-12).
- One bureau key posts grades (KR-1); upgrades take effect only after the on-chain upgrade delay and every posting is publicly recomputable.
- The demo profile shortens the age gate to 600 s (production: 30 days).

Deviations from the specification text (recorded for review)

1. **F6 volume spike threshold.** Spec 4.3 computes the spike "if at least 6 exist", but golden vector G8 (Atlas with five 0.20 orders and one 0.80 order, score 760, reasons 1006, 1007) requires no spike penalty with exactly six payee orders. The golden table wins: the spike is computed only when at least `spike.min_prior_orders` (4) payee-role orders precede the last `spike.window_last` (3), i.e. with seven or more payee-role orders. The key is renamed from the spec's `min_orders: 6` so the config says what the model does. Every other golden vector is unaffected.
2. **Highest P in a cluster (F5a).** "The highest P in that cluster" is read as the highest standing among the cluster's members that were counterparties in a settled counted order with the subject (the more restrictive reading; identical on every golden vector).
3. **Circular volume graph.** Nodes of the payment graph are clusters, so a ring that routes through two agents of one owner is still circular (more restrictive; identical on every golden vector).
4. **Bundle `model.anchors`.** The bundle's `model` object also lists the anchors used (spec 13.3: "every bundle lists the anchors it used").
5. **Order timestamp for decay.** A failure decays from the time of its closing event (refund or recorded delivery).
6. **First postings (spec 4.10).** Every agent of the grading universe is scored, but an agent is posted for the first time only when it is listed in `deployments/<network>.json` or has evidence of its own (a counted order or a bound report). Any registered vault can name payees for free (`recordDecision`, `ownerApprove`), so without this bound such names could drain the bureau key and put feedback on unrelated agents.

Version history

Version	Date	Change
acb-sc-1.0.0	2026-10-12	First version (spec v1.1, section 4)